开发内环与CANN算子开发体验

云开发环境和一站式CANN开发体验
助力昇腾算子开发加速

华为开发者空间首席架构师 Eric

从“门洛帕克实验室”讲起

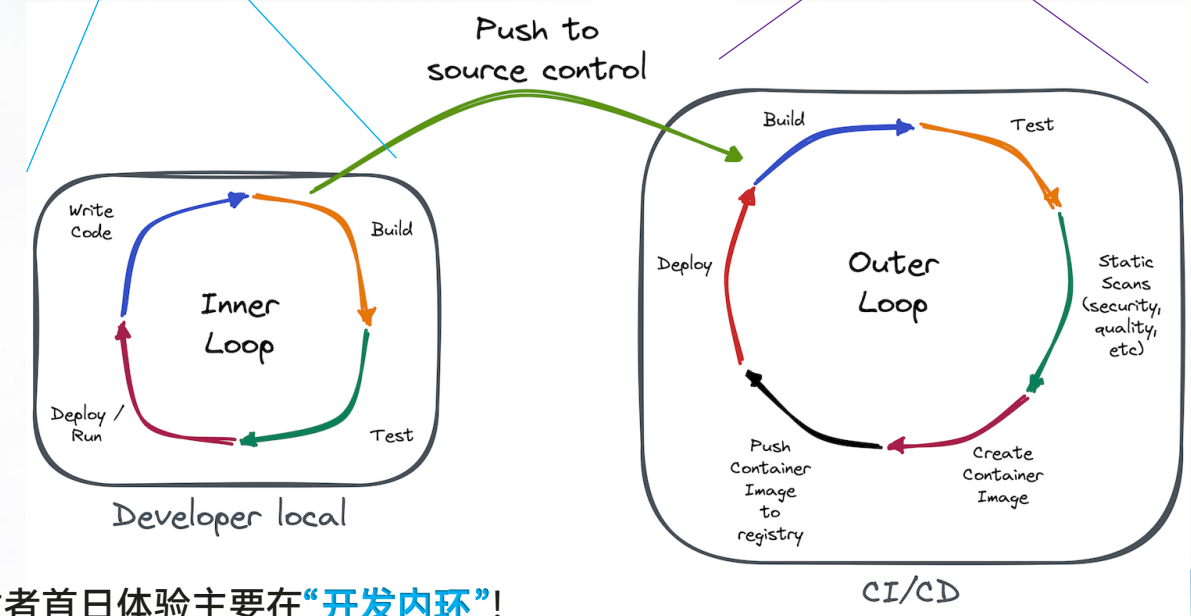
- 诞生了世界上第一个电灯泡
- 这个实验室在18个月的大部分时间里试验了 6,000+ 的灯丝候选材料
- 在最后2个月完成了真空封装、插座兼容性、城市规模试验等工作

什么是“开发内环”(Dev Inner Loop）？

1878/4 ~ 1879/10
试验各种灯芯材料(内环)

1879/10 ~ 1879/12
封装集成测试(外环)

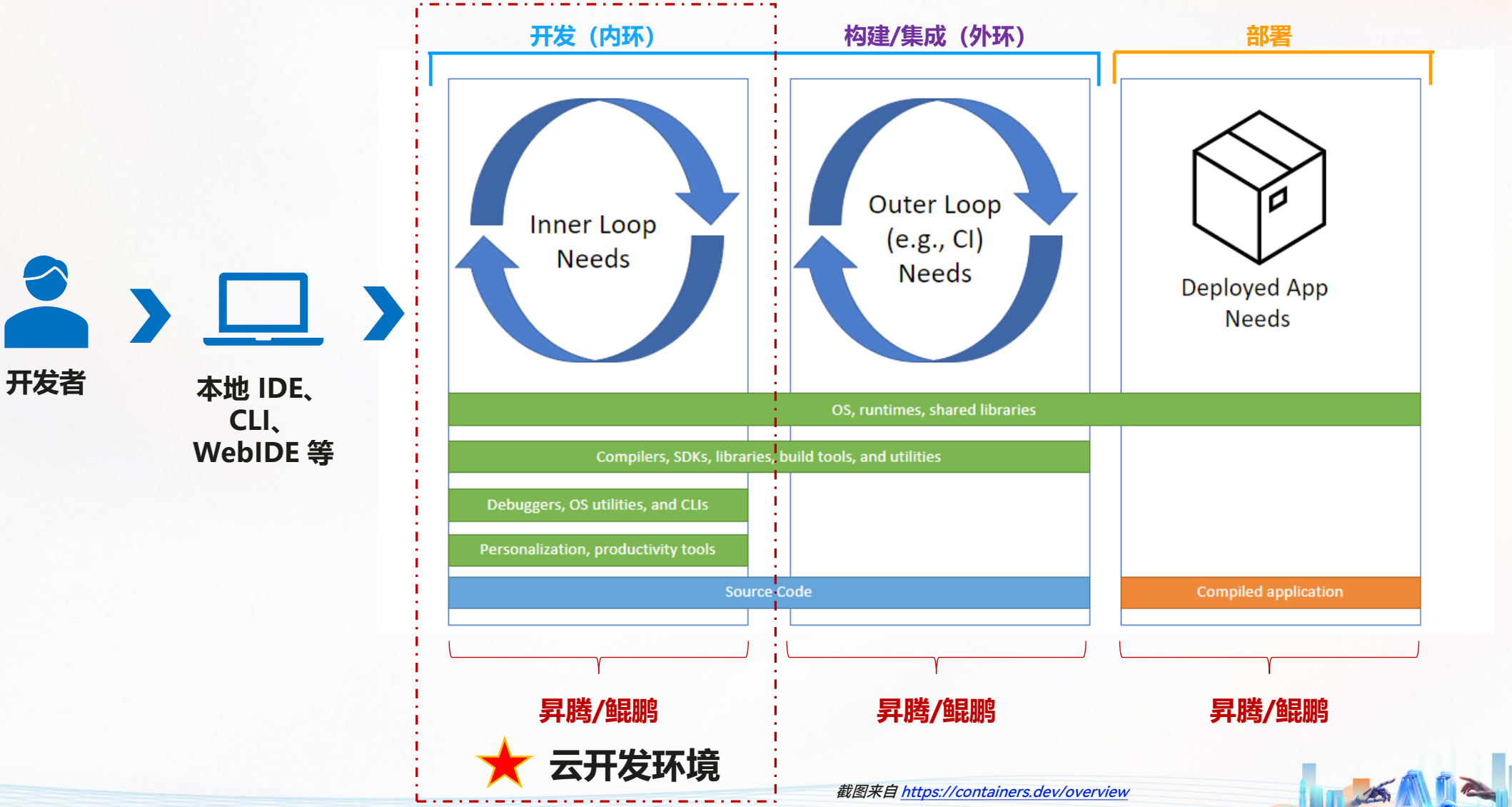
- 几乎所有软件开发均为迭代式。
- 开发内环指开发者在提交代码前的本地操作：搭建和配置开发环境、原型试验、编码构建、测试及调优，可能包含针对同一功能的多次Git提交。
- 当代码进入需自动化构建、测试、扫描及部署的阶段时，即过渡至开发外环(CI/CD流程）。



★ 开发者首日体验主要在“开发内环”!

截图来自于 https://developers.redhat.com/articles/2022/12/12/kubernetes-native-inner-loop-development-quarkus

云开发环境是解决开发者“内环”开发体验的重要工具



截图来自 https://containers.dev/overview

CANN 生态下的“内环”：通过云开发环境提升资源的获取便利和体验

较传统生态，CANN 生态开发者**对获取+访问鲲鹏、昇腾算力的友好性更期望**。

开发者空间和开源仓库社区的合作，在开源项目增加**一键拉起云开发环境**的入口，对接昇腾算力，帮助开发者**快速完成“内环”开发任务**。

开发者空间的云开发环境

桌面版

传统桌面开发场景

举例：鸿蒙 PC开发场景

传统服务器类开发场景

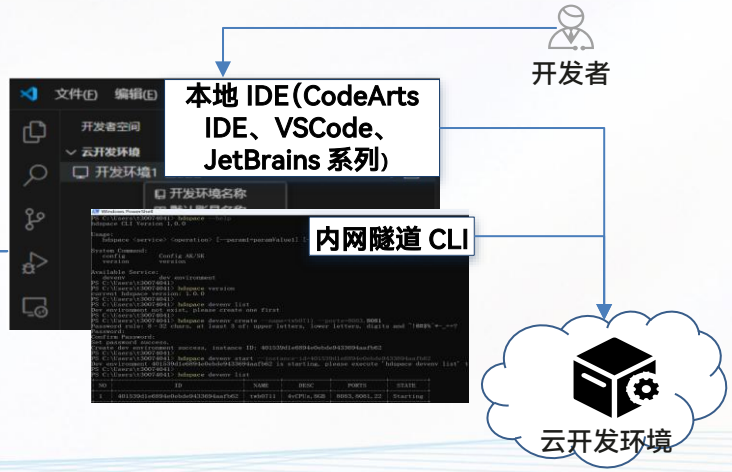
举例： Euler/ OpenEuler 服务器开发场景

虚拟机版

容器版

热门Docker容器开发场景

举例：鲲鹏/昇腾算子容器化开发场景



桌面版:

- 支持通过本地电脑**远程桌面**访问鲲鹏/昇腾环境
- **预装**好 CodeArts IDE 等开发所需工具
- 适合图形化工具开发场景

虚拟机版:

- 支持本地 IDE(如 CodeArts IDE、VSCode、JetBrains）、CLI 等工具**直连**鲲鹏/昇腾环境
- 基于欧拉OS，提前**预装**好相关工具，**开箱即用**
- 适合 OS 层、内核应用、容器编排等相关的开发场景

容器版:

- 支持本地 IDE(如 CodeArts IDE、VSCode、JetBrains）、CLI 等工具**直连**鲲鹏/昇腾环境
- 直接使用用户指定镜像，**快速**拉起开发环境
- 支持**动态交付及扩缩容**，**灵活**管理开发测试预算
- 适合上层业务应用等开发场景

来自社区对CANN算子开发内环的用户声音

算子开发工程师的“内环”旅程：

感知学习	环境准备	算子开发	算子编译	功能调试	性能调优
01	02	03	04	05	06

开发者用户原声：

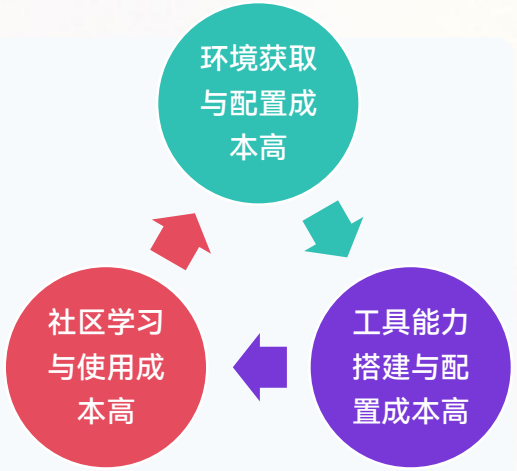
希望提供本地IDE连接一站式开发平台的能力，保持本地IDE开发习惯。

希望开发环境可以保存我的开发的代码，关机后不要丢失。

想尝试基于下一代硬件开发，希望有对应的开发环境。

能支持算子精度和性能调优。

提供使用Python语言开发算子的环境。

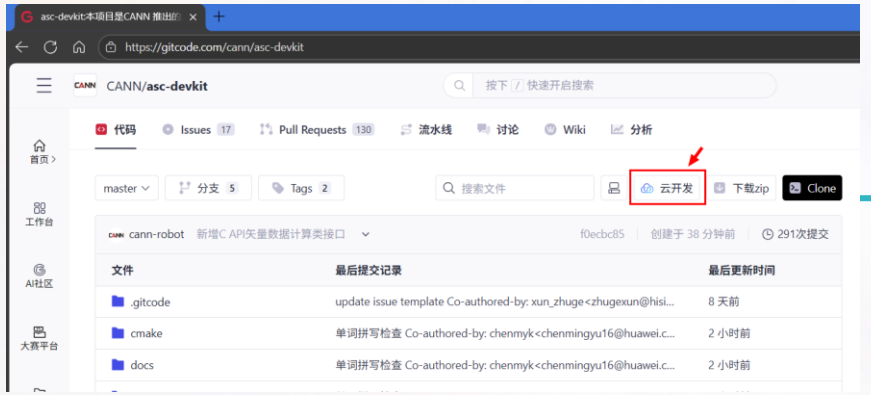


开发者用户需要的不只是更丰富的工具和步骤，而是更完整的“开发内环”体验！

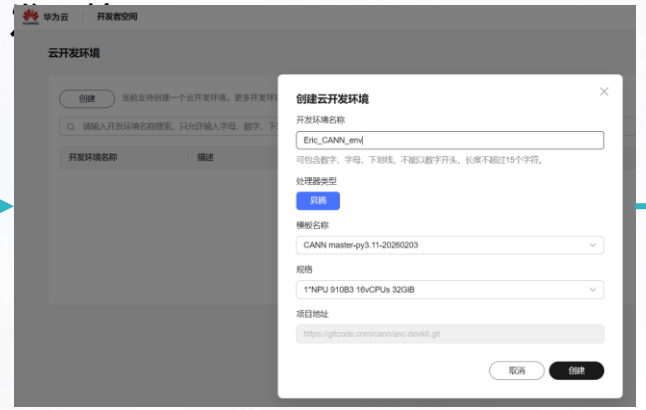
从 GitCode 开源社区一键拉起 CANN 一站式开发平台

解决算子开发效率低，开源社区缺少一站式开发环境，开发者参与门槛高。开发者空间通过云开发环境提供构建算子一站式开发平台以及算子开发调优的开箱即用能力，包含开箱即用开发环境、版本配套容器环境、编码IDE插件。使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态。

1、开发者访问CANN开源项目，点击云开发



2、进入开发者空间引导页，创建云开

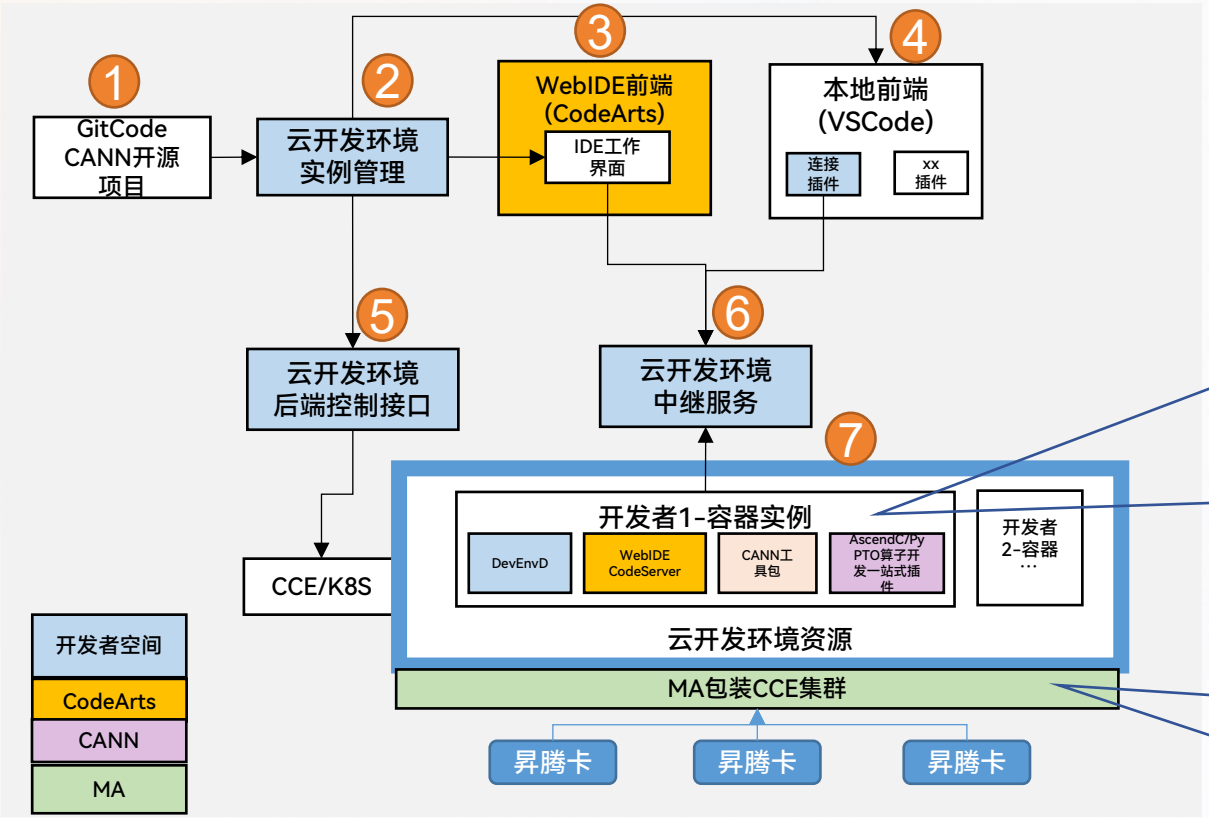


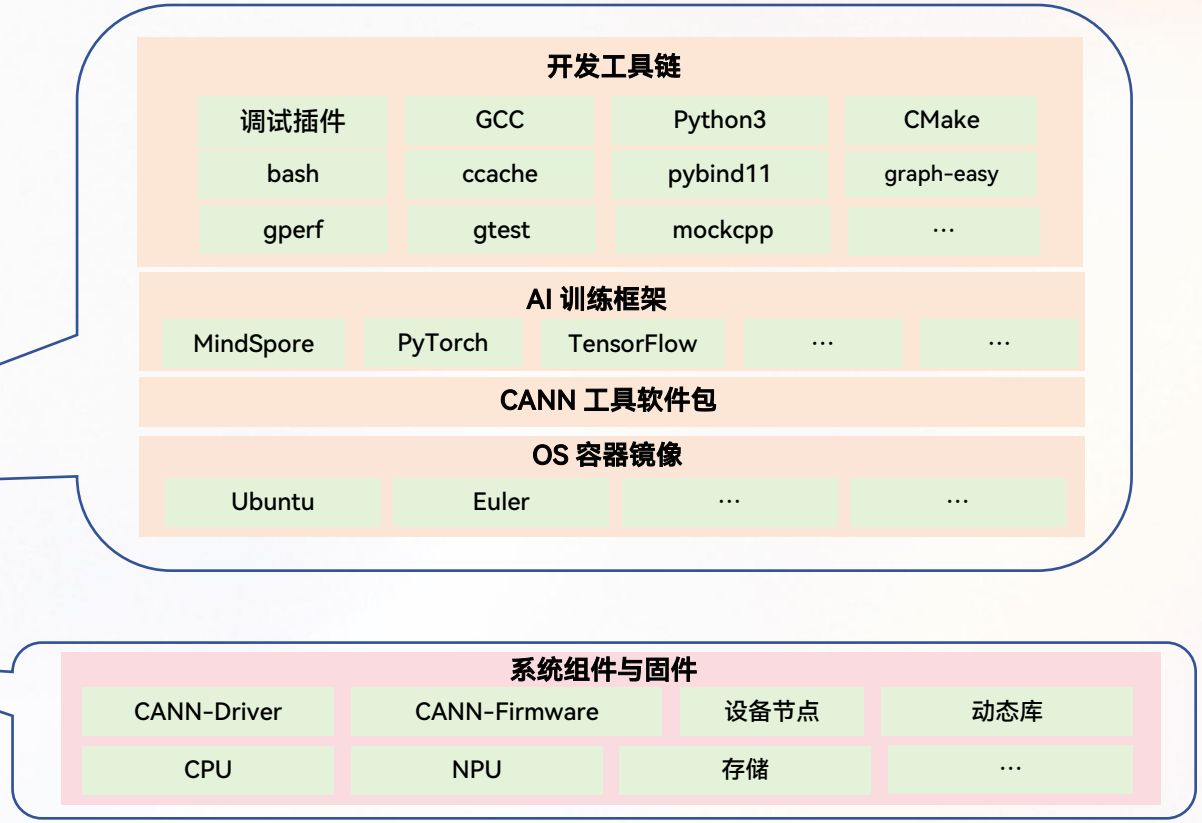
3、云开发环境默认自动开机，点击连接

4、进入 CodeArts IDE 页面，即 CANN 算子一站式开发平台

CANN算子开发样例：https://gitcode.com/cann/asc-devkit

CANN 一站式开发平台 Inside Out





1、**开发者空间**：提供云端开发环境算力服务调度及管理，以及隧道反代连接能力，帮助端侧直达云端昇腾算力环境。

2、**ModelArts**：提供昇腾卡云化能力，将物理卡转化为云上可调用的昇腾云服务。

3、**CANN 团队**：提供CANN工具包及AscendC/PyPTO算子开发插件。

4、**CodeArts**：提供 WebIDE 前端工作页面。

提供集成的开发工具和开箱即用的昇腾算力，快速上手参与社区贡献

代码贡献者 | Issues 管理者 | 社区运营者 | 技术体验者 | 生态伙伴

触达
开发者指尖

IDE

社区

CANN

GitCode

汇聚
开发者智慧

一站式开发平台

工程创建 → 算子编写 → 算子调试 → 异常检测 → 性能调优(上板) → 性能调优(仿真)

WebIDE/本地IDE

- **在线一站式**编码调试调优
- **本地IDE**对接，支持离线编码

算子开发能力

- 支持 pyAsc **算子调试调优**，支持精度校验、pyPTO IR控制流
- **AI 辅助**算子开发/补齐/续写

云开发环境

- 用户**数据持久化**保存，存算分离
- IDE/Web/CLI 直连，**无需公网IP**

算力模板配置与运营

- 不同场景、人员灵活配置模板
- 团队声明式定义、可视化管理

社区流水线体验

代码托管 → PR触发 → Fork PR门禁 | PR评论触发

GitCode
CodeArts Pipeline

流水线代码化

流水线配置下沉到代码仓，支持开发者本地搭建流水线

构建效率优化

- 非缓存场景下kernel编译时长<40min
- 组合出包，构建耗时<1h

测试能力建设

- 外溢测试框架/测试工具/测试用例(RDV)
- 外溢黄区测试策略/验证工具(版本级)

发布

- 持续完善各社区官方仓，构建统一/安全/高性能的多语言仓库基础设施

GitCode 账号统一

注册(GitCode + 华为云) → 认证(华为云) → 登录

统一社区登录账号，一个账号全流程开发作业

注册登录体验优化，断点消除

算力资源统一管理

- 基于华为云，建立CANN、昇腾、鸿蒙统一的算力资源大池，实现资源按需扩缩容。

AI
CANN

CANN Simulator仿真工具

CANN社区 Committer: 刘俊

2026.2.12

CANN Simulator是什么？

CANN Simulator是面向下一代芯片推出的一款算子开发的仿真工具，
让用户不依赖NPU设备基于X86/ARM服务器进行算子开发

- 支撑开发人员完成算子的功能验证、精度分析
- 定位算子执行瓶颈，优化指令排布等

随时随地开发

- HIF8/SIMT等特性算子功能验证
- 探索新芯片对于算子性能的提升空间

尝鲜新特性

- 接近真实设备功能、精度和性能仿真能力，支撑用户代码可零修改迁移到NPU环境

零修改迁移

- 提供比上板更详细的指令级流水分析

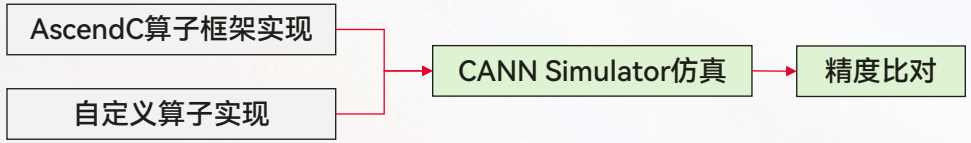
可视化性能分析

CANN Simulator核心能力：精度和性能调优

支持关键新特性仿真

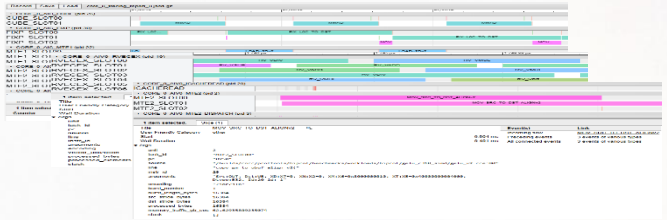
特性类型	支持程度
运行平台	X86/ARM
指令集类型	SIMD、**SIMT**
数据类型	**FP4**、**FP8**、**HiF8**、**MXFP8**、FP16、FP32、INT、BF16

精度分析：工具输出仿真的结果，供算子进行精度的比对



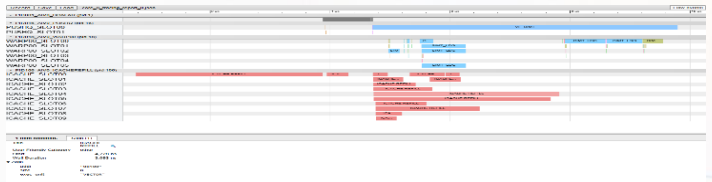
性能调优：根据仿真结果，分析存在的性能问题。（结合算子代码、指令流水图、CA Model的log）

① **过程信息可视化：**通过工具的可视化展示，用户可以更清晰地看到Memory、Vector、Cube指令流水详细指标，内部的缓存队列中间状态，方便进行性能分析和优化



② **快速找到性能瓶颈代码行：**通过提供源代码与汇编代码的映射，以及可视化同步指令的依赖关系，用户可以更精确地识别性能瓶颈，提升代码执行效率。

③ **更便捷的定界定位能力：**工具提供的iCache视图，可以快速识别缓存命中/非命中场景下对系统性能的影响，并据此调整算法，从而提升整体性能。



（部分功能预计在26年H1开放）

CANN Simulator工具使用流程

01-工具安装

- CANN Simulator集成在CANN软件包里，使用工具前，需要安装CANN软件包(不依赖驱动固件），详细请参考《CANN软件安装指南》。安装完成后执行执行source ${INSTALL_DIR}/bin/setenv. bash命令设置环境变量。

02-编译算子

- 完成算子编译工作，详细请参见《Ascend C算子开发指南》中的“编译与运行 > 算子编译”章节。

03-执行仿真

- 执行仿真命令，采集仿真数据:

  cannsim record -s ascend950 -o $HOME/projects/output $HOME/projects/MyApp/mainapp -- mainapp文件是算子直调的可执行文件

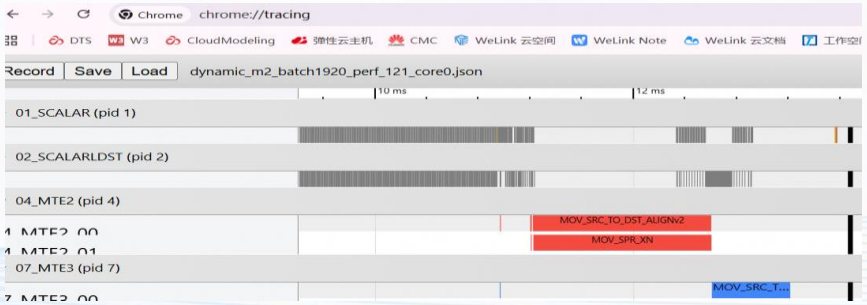
04-结果分析

- 执行仿真结果解析命令，分析仿真数据。

  cannsim report -e $HOME/projects/output/cannsim_{timestamp}_${user_app} -o .

05-结果查看

- 仿真结果查看，Chrome浏览器中输入“chrome://tracing”地址，并将通过生成指令流水图文件(trace.json）拖到空白处打开



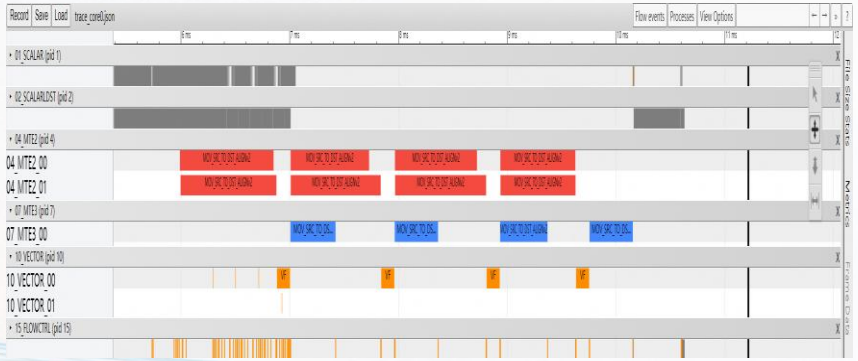
如何使用CANN Simulator进行性能调优?

典型问题：指令流水排布不合理

运行CANN Simulator仿真

1、问题定位

查看算子指令流水图，从图中可以发现MTE2流水在VADD计算时，没有执行搬运指令，且MTE2流水为该算子的性能瓶颈，需提高MTE2的搬运效率以实现算子性能优化



2、修复问题

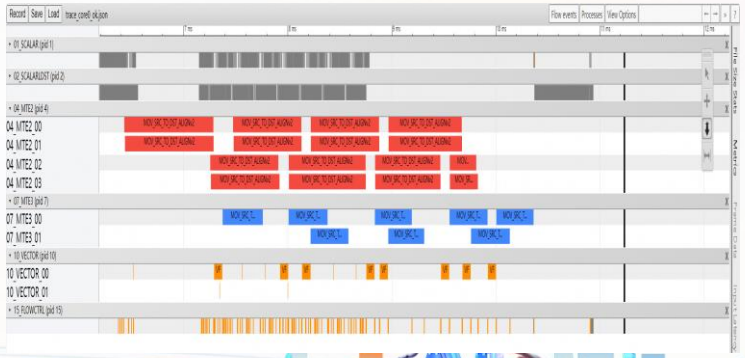
对于MTE2搬运效率的提升有多种方式，可通过将TPipe中InitBuffer的第二个参数（BUFFER_NUM）值从1修改为2，开启double buffer

```
constexpr int32_t BUFFER_NUM = 2; // tensor num for each queue
...
pipe.InitBuffer(inQueueY, BUFFER_NUM, 1024 * sizeof(half));
...
```

3、结果查看

重新执行，查看优化后的指令流水图。

在VADD指令计算时，MTE2上的搬运指令也同步执行，实现了更高效的数据搬运



CANN Simulator工具支撑范围

CANN Simulator仿真工具推荐系统配置：CPU 16U及以上，内存32GB及以上

支撑范围

CPU类型	是否支持
X86	支持
ARM	支持

算子类型	是否支持
AICore算子	支持
AICpu算子	敬请期待
通信类算子	敬请期待

指令集类型	是否支持
SIMD	支持
SIMT	支持

数据类型	是否支持
FP8	支持
FP4	支持
FP16	支持
FP32	支持
INT	支持
BF16	支持
HiF8	支持

AI
CANN

【彩蛋】CANNBot Agent

CANN算子优化专家: 金炯

2026.2.12

- 一、CANNBot Agent介绍

CANNBot Agent介绍

大象装进冰箱

1. 打开冰箱门
2. 把大象放进去
3. 关上冰箱门

CANNBot做算子开发

1. 描述你的需求
2. Agent自动规划
3. 自动执行完成

CANNBot是专门为CANN算子开发准备的AI Agent小助手。

CANNBot Agent架构

Agent交互平台

负责交互/规划/执行

LLM

负责智能理解, 用户可配置

CANNBot领域知识

SKILLS

- ascendc-docs-search
  高效的文档检索
- operator-precision-debug
  算子精度调试技巧
- operator-performance-optimization
  算子性能调优

WorkFlows

- ascendc-kernel-develop-workflow
  kernel直调开发流程
- operator-develop-workflow
  算子开发流程
- operator-auto-generate
  算子自动生成

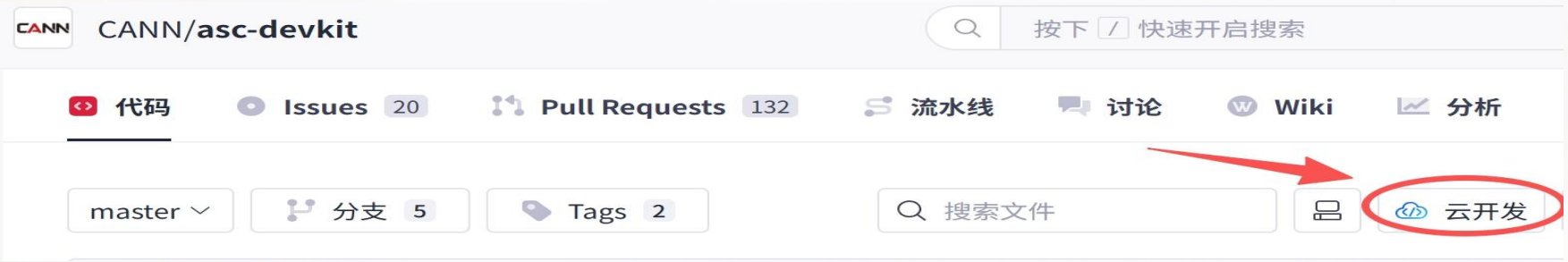
Docs/Tools

- ascendc-api-docs
  Ascendc基础API文档
- msprof-docs
  性能工具使用文档
- env-install-docs
  环境安装文档

- 二、CANNBot Agent使用说明

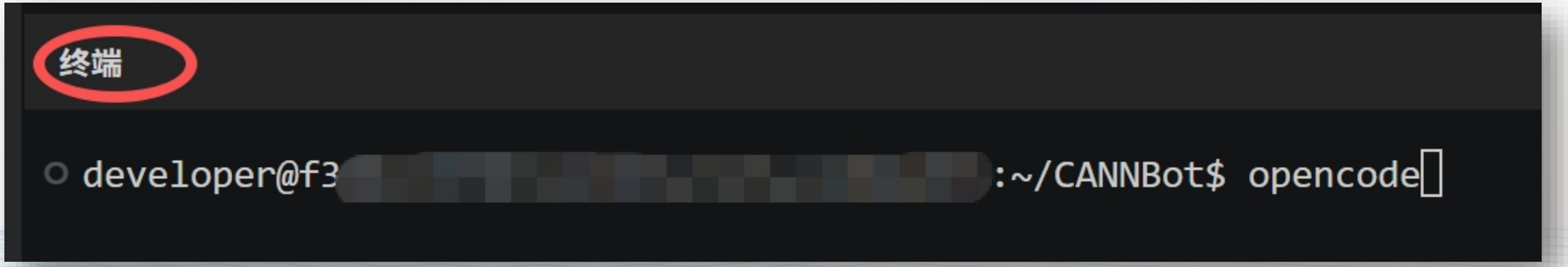
步骤1：启动Agent工具

- 从asc-devkit仓进入一站式开发平台



- 以WebIDE方式登陆云开发环境

- 打开”终端”, 敲 opencode (已预装)进命令行



📖步骤2：配置API Key

- 输入/connect选择模型，用户可配置自己API Key，本次演示我们使用的是 GLM 4.7

- 没有API Key的开发者，也进入开发者空间，在激励管理中申请试用

1 场景演示：说 Hello

输入指令：

```
你好！

Build  GLM-4.7 Zhipu AI Coding Plan
```

输出结果：

```
你好！我是 CANNBot，专门帮助你在华为昇腾 AI 处理器上开发 Ascend C 算子。

有什么我可以帮你的吗？

▣ Build · glm-4.7 · 5.2s
```

✨ 第一次交互成功！CANNBot 已就绪。

https://gitcode.com/cann

2 场景演示：查找特定 API 的用法

输入指令：

```
帮我查一下ReduceMax的用法

Build  GLM-4.7 Zhipu AI Coding Plan

                                  tab agents  ctrl+p commands
```

输出结果：

```
- 基础示例：asc-devkit/examples/03_libraries/05_reduce/reducemax/
- WholeReduceMax：asc-devkit/examples/03_libraries/05_reduce/whole_reduce_max/
- BlockReduceMax：asc-devkit/examples/03_libraries/05_reduce/block_reduce_max/

关键注意事项

- 地址对齐：src 和 sharedTmpBuffer 需 32 字节对齐
- 数据类型：支持 half/float（A3 环境）
- mask 模式：支持逐 bit 模式和连续模式
- 索引读取：half 类型用 reinterpret_cast<uint16_t*>，float 类型用 reinterpret_cast<uint32_t*>

详细 API 文档：asc-devkit/docs/api/context/ReduceMax.md

▣ Build · glm-4.7 · 3m 24s

Build  GLM-4.7 Zhipu AI Coding Plan
```

自动启用 ascendc-docs-search SKILL 高效检索文档

3 场景演示：执行样例

输入指令：

```
执行asc-devkit/examples/00_introduction样例

Build  GLM-4.7 Zhipu AI Coding Plan

                                    tab agents  ctrl+p commands
```

输出结果：

```
Thinking: 太好了！样例执行成功。输出了8个 "Hello World!!!"，每个 Block 打印一个。这符合预期的结果。

执行成功！HelloWorld 样例已正确运行，8个 Block 各打印一条 "Hello World!!!"。

Build · glm-4.7 · 53.0s

Build  GLM-4.7 Zhipu AI Coding Plan
```

一键运行样例，快速验证功能

4 场景演示：简单算子生成

输入指令：

```
帮我生成一个abs算子

Build  GLM-4.7 Zhipu AI Coding Plan
```

输出结果：

```
状态：✓ 全部通过

验证项目

| 项目 | 结果 | 说明 |
|------|------|------|
| 构建状态 | ✓ PASS | 可执行文件存在 |
| 功能测试 | ✓ PASS | 589,824元素验证通过 |
| 精度验证 | ✓ PASS | 容忍度0.001内全部正确 |
| 双缓冲流水线 | ✓ 正确 | Ping-pong机制实现正确 |
| 多核并行 | ✓ 正确 | 16核并行配置 |
| API使用 | ✓ 规范 | 使用AscendC::Abs()标准API |

代码质量检查
```

资源管理器

- ABS
  - build
  - input
  - output
  - abs.asc
  - CMakeLists.txt
  - data_utils.h
  - generate_data.py
  - generate_report.py
  - run_test.sh
  - test_report.md
  - verify_output.py

abs.asc

```
37  public:
44      }
45
46      __aicore__ inline void Process()
47      {
48          AscendC::LocalMemAllocator<AscendC::Hardwar
49          AscendC::LocalTensor<float> xLocalPing = ub
50          AscendC::LocalTensor<float> yLocalPing = ub
51          AscendC::LocalTensor<float> xLocalPong = ub
52          AscendC::LocalTensor<float> yLocalPong = ub
53
54          AscendC::SetFlag<AscendC::HardEvent::MTE3_M
55          AscendC::SetFlag<AscendC::HardEvent::MTE3_M
56          for (uint32_t i = 0; i < loopCount; i++) {
```

```
Build  GLM-4.7 Zhipu AI Coding Plan
```

一键生成简单算子，出具测试报告

演进计划和路标

本次一站式平台将预装CANNBot尝鲜版，已装备文档搜索、kernel直调开发流程等技能。

Q1上线 SKILL

单元测试
单元测试生成与执行

集成测试
自动化集成测试

资料开发
文档自动生成与优化

代码检视
代码检视与修复

Q2~Q3上线 SKILL

精度调试
精度诊断 | 渐进调试
tensor打印 | 仿真调试

性能调优
性能验证 | 流水分析
理论建模 | tiling设计

复杂算子生成
复杂任务规划
功能 | 性能模板拆分

更多编程语言, 如pypto
更多平台接入
面向下一代芯片的技巧

🌟 敬请期待：CANNBot让算子开发更简单！

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯